Low-Level Design (LLD) Practice Platform MVP

A focused, end-to-end Low-Level Design (LLD) practice platform built for software engineers to practice object-oriented modeling, submit multi-faceted design solutions, receive explainable rubric-driven feedback (deterministic + AI reasoning), and track attempt progression over time. ---

🌟 Key Features

1. **Curated Problem Catalog:** Realistic LLD challenges (Parking Lot System, Elevator Controller, Vending Machine with State Pattern, Distributed Rate Limiter & LRU Cache). 2. **Multi-Tab Design Studio:** Write code (`TypeScript` / `Java`), define Class Responsibility Mappings, document Trade-offs & Concurrency Assumptions, and sketch Mermaid Class Diagrams. 3. **Hybrid Evaluation Engine:** Combines **deterministic static analysis** (structure, signature coverage, god-object anti-patterns) with **LLM architectural reasoning** (SOLID principles, coupling, extensibility). 4. **Resilient Offline Fallback:** Fully operational with or without a Gemini API key via an intelligent heuristic architectural reasoner fallback. 5. **Explainable Feedback Dashboard:** Score breakdown across 6 rubric axes, specific line/concept evidence citations, strengths vs anti-patterns, and extensibility stress-test hints. 6. **Attempt Progression & Comparison:** Track attempt history (v1 $\rightarrow$ v2) and compare attempts side-by-side. ---

🚀 Quick Start (Single Command Run)

Prerequisites

- **Node.js**: v18+ (Tested on v22.22.2)
- **NPM**: v9+

1. Installation

``bash

Navigate to the project folder

```
cd /Users/piyushraj/.gemini/antigravity-ide/scratch/lld-practice-platform
# Install root & backend dependencies

npm install
# Build client React application bundle

npm run client:build `
```

2. Launch Platform

```bash
# Start the full-stack server (Backend API + Frontend UI on single port)

npm start ` Open http://localhost:4000 in your browser to experience the platform! ---
```

🧪 Running Automated Tests

The repository includes a unit and API test suite covering domain entities, state machine transitions, evaluator rules, and REST endpoints: ` bash npm test ` Sample Test Output: ` text PASS tests/domain.test.ts LLD Platform Domain Core Tests ✓ Submission correctly parses lines of code and extracts declared classes ✓ Attempt enforces valid state transitions and guards against illegal transitions ✓ DeterministicEvaluator assesses domain concept coverage and interfaces ✓ LLMEvaluator fallback reasoning operates resilience without external API key ✓ EvaluationPipeline aggregates evaluations into FeedbackReport REST API Endpoints ✓ GET /api/problems returns list of problems ✓ POST /api/attempts creates attempt and starts async evaluation Test Suites: 1

passed, 1 total Tests: 7 passed, 7 total ` ---

📂 Project Architecture

` text lld-practice-platform/ ├── src/ # Backend Domain Core & Express API | ├── domain/ # Domain Entities & Value Objects | | ├── Problem.ts # Problem & Rubric entity | | ├── Submission.ts # Multi-part Submission value object | | ├── Attempt.ts # Attempt aggregate root (State Machine) | | ├── Rubric.ts # 6-Axis Rubric specifications | | ├── FeedbackReport.ts # Explainable feedback report | | └── evaluators/ # Strategy Pattern Evaluators | | ├── Evaluator.ts # IEvaluator interface | | ├── DeterministicEvaluator.ts # Static analysis checker | | ├── LLMEvaluator.ts # Gemini API / Heuristic fallback reasoner | | └── EvaluationPipeline.ts # Composite evaluator manager | ├── repository/ # Problem & Attempt repositories (JSON disk persistence) | ├── services/ # EvaluationService (Async orchestrator) | ├── controllers/ # REST Endpoints | ├── app.ts # Express app configuration | └── server.ts # Server entry point ├── client/ # Interactive Frontend UI (Vite + React + TS) | ├── src/ | | ├── components/ # Catalog, Studio, Feedback, History Modal | | ├── App.tsx # Main SPA router & polling logic | | └── index.css # Glassmorphism dark design system ├── tests/ # Jest domain & API test suite ├── RESEARCH.md # 1-2 pages research note on LLD learning gaps ├── DESIGN.md # System design note & Change Test A/B proofs ├── AI_USAGE.md # 3-5 key AI-assisted decisions & rationale └── README.md # Documentation ` ---

🔑 Optional Gemini API Integration

The platform runs 100% offline out-of-the-box using built-in heuristic reasoning. To enable live Gemini API architectural evaluations: ` bash export GEMINI_API_KEY="your-gemini-api-key" npm start ` ---

📄 Submitted Deliverables Checklist

- [x] **Research Note (RESEARCH.md)** : 1–2 pages on learner problem, existing tools, gaps, and product thesis.
- [x] **Design Note (DESIGN.md)** : Concise explanation of MVP, user flow, class responsibilities, evaluation approach, and Change Test A/B proofs.

- [x] **Working Prototype:** End-to-end web practice studio with async evaluation and attempt history.
- [x] **Tests (`tests/domain.test.ts`):** 100% passing test suite for core domain behavior, state transitions, evaluators, and API endpoints.
- [x] **AI Usage (`AI_USAGE.md`)**: Documented 4 key AI-assisted decisions, proposals rejected, and trade-offs.
